TT2026-S2-N2 · PROYECTO FINAL

TeamUp ZK

Anonymous team formation on Ethereum with Semaphore zero-knowledge proofs

A protocol for course team formation where membership is proven in zero knowledge, applications are keyed by nullifier instead of wallet, and accepted members claim an on-chain NFT credential.

NETWORK

Sepolia · chainId 11155111

STACK

Solidity · Hardhat · Semaphore V4 · Next.js

REPOSITORY

github.com/0xAstraea/eth-kipu-final-project

DATE

October 2026

Summary

TeamUp ZK is an on-chain protocol for course team formation. The teacher controls who joins the course group. A student proves membership with a Semaphore zero-knowledge proof and applies **without revealing who they are**. The credential is an NFT. The student claims it to any address, including a throwaway one.

The MVP covers: enroll, create team, anonymous apply, accept, reveal roster, claim NFT.

1. Problem

Course teams often use a shared spreadsheet. That fails in three ways:

1. **No membership check.** Anyone can write any name.
2. **Public interest.** Seeking a team can carry a social cost.
3. **No verifiable record.** Agreements live in chat or cells. No signed history.

2. Why blockchain

A central server can only *promise* the rules. A contract *enforces* them. Anyone can check state and events.

Need	Off-chain	On-chain (TeamUp ZK)
Only course members apply	Trust the operator	ZK proof against the course group
No double apply	Easy with a new name	Nullifier burned per (identity, team)
Hide who applied	Server can log and correlate	Contract never sees the identity
Final roster + credential	Editable file	`rosterRoot` + NFT after `reveal`

The teacher still owns the roster. Chat and skill matching stay off-chain. The chain does not decentralize the classroom. It makes join, anti-Sybil, reveal, and credential rules independent of operator good faith.

3. Goals and threats

Goals

- Only course members create teams and apply.
- An application does not bind to `msg.sender`.
- One identity applies once per team.

- The team accepts; one credential per member.
- Events make the flow auditable.

Threats

Adversary	Defense
Non-member	Invalid proof → revert
Sybil (same identity twice)	Repeated nullifier → revert
Observer	Events show nullifiers, not addresses
Claim front-run	NFT always mints to the committed address

Out of scope (MVP): mempool griefing (see §10), economic incentives.

4. Architecture

Contract	Role
`Semaphore` (PSE)	Course group + ZK verification
`TeamRegistry` (ours)	Teams, applications, reveal, claim
`TeamCredential` (ours, ERC-721)	Membership NFT

`TeamRegistry` **does not inherit** `Semaphore`. It calls `ISemaphore`. If it inherited, anyone could call public `addMember` and bypass the teacher. The registry is the group admin. Only `addCourseMember(s)` (owner) enrolls students.

Own contracts link through `ITeamCredential`: registry mints; credential accepts only its fixed registry.

5. Protocol flow

1. **Enroll.** Owner calls `addCourseMember(s)` with identity commitments.
2. **Create team.** Member submits ZK proof (`scope = teamId`) via `createTeam`.
3. **Apply.** Member submits proof (`message = keccak256(presentation, claimCommitment)`) via `requestJoin`. Storage key = nullifier. No `msg.sender` stored.
4. **Accept.** Team admin calls `acceptRequest(teamId, nullifier)`.
5. **Reveal.** When full or `block.number ≥ revealAtBlock`, anyone calls `reveal`. Contract freezes the roster, emits sorted nullifiers, stores `rosterRoot = keccak256(sortedNullifiers)`.
6. **Claim.** After reveal, `claimCredential` checks `keccak256(claimAddress, salt)` and mints the NFT to `claimAddress`. Anyone may pay gas.

6. Cryptography

- **Membership proof.** Semaphore v4 / LeanIMT. Proves knowledge of a commitment in the tree.
- **Nullifier.** Stable id for accept/claim. Blocks a second apply. `validateProof` burns it.
- **Authenticated message.** `keccak256(presentation, claimCommitment) mod SNARK_FIELD_SIZE`. A third party cannot change either field.
- **Claim commitment.** `keccak256(abi.encode(claimAddress, salt))`. Uses keccak256, not the experimental Semaphore ownership circuit.

7. Security

- **CEI.** `claimCredential` sets `claimed = true` before mint.
- **ReentrancyGuardTransient** on functions with external calls.
- **Access.** Ownable for enroll/ `setCredential`; team admin for accept; credential `mint` only from registry.
- **Custom errors** and **events** on every state change.
- No `tx.origin`, `delegatecall`, or `selfdestruct`.
- Slither triage: `security/slither-report.md`.

8. Gas

- `immutable` / `constant` for Semaphore, group id, field size, credential registry.
- Pack `admin` + `maxMembers` + `revealAtBlock` in one slot.
- Revealed state = `rosterRoot ≠ 0` (no extra bool).
- `unchecked` on safe counters; `calldata` for external args; transient reentrancy guard.
- Batch enroll via `addCourseMembers`.
- Reuse Semaphore stack across deploys.
- Numbers: `GAS-OPTIMIZATION-REPORT.md`.

9. Tests

- **46** contract tests: happy path + main reverts.
- **Coverage:** 100% lines and statements on own code (`pnpm run coverage`).
- **16** frontend tests on shared hashing.
- Slither on `TeamRegistry` and `TeamCredential` (deps filtered).

10. MVP limits

1. **Statistical anonymity.** Groups of 1–2 members are not anonymous (Semaphore docs).
2. **Tx sender is public.** A relayer is needed to hide `msg.sender` on the network.
3. **Mempool griefing.** Copying a proof can burn the victim nullifier. Mitigate with private relays.
4. **No incentives.** No stake or slash for leaving a team.

11. Roadmap

Phase	Item	Why
2	Staking / slashing	Cost for abandoning a team
2	Relayer	Hide sender; stop mempool griefing
3	Blind recruit	Accept by nullifier; reveal at claim (stable ownership circuit)
3	Cumulative reputation	Participation credentials without public scores
4	Matching off-chain	Skills/roles via subgraph events

Web UI: delivered (`frontend/`).

12. Stack

Solidity 0.8.28 · Hardhat 3 · ethers v6 · OpenZeppelin 5 · Semaphore V4 · TypeScript (ESM) · Mocha · Slither · Next.js 16 · wagmi 3.